DOCS_SIMILARITY

Document Similarity Analyzer Using Suffix Arrays and Kasai's LCP Algorithm

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by
Billakanti Siri - Roll No.: 2520030601
Bhavana Kondakrindi - Roll No.:2520030082
Aniketh Pani- Roll No.:2520030182

Team Number: 7

Section: 1

Under the guidance of

Dr.Ch.Anuradha

Professor, CSE



Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING / COMPUTER SCIENCE AND INFORMATION TECHNOLOGY/ARTIFICIAL INTELLIGENCE & DATA SCIENCE



DECLARATION

We hereby declare that the project report entitled "DOCS_SIMILARITY: Document Similarity Analyzer Using Suffix Arrays and Kasai's LCP Algorithm" is a record of bona fide work carried out by the undersigned students in partial fulfilment of the requirements for the Data Structures and Algorithms–3 course at K L (Deemed to be) University.

We further declare that the work presented in this report is our project work and that all external sources consulted have been acknowledged in the references. The implementation details and results described in this report should be read in conjunction with the submitted source code and actual application output.

Billakanti Siri - Roll No.: 2520030601
Bhavana Kondakrindi - Roll No.:2520030082
Aniketh Pani- Roll No.:2520030182

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING / COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the mini project based report entitled "**DOCS_SIMILARITY: Document Similarity Analyzer Using Suffix Arrays and Kasai's LCP Algorithm"** is a bonafide work done and submitted by **Billakanti Siri-2520030601, Bhavana Kondakrindi-2520030082, and Aniketh Pani-2520030182** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my guide, Dr. Ch.Anuradha, Professor, CSE which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P.Venkateshwar Rao (CSE)**, Head of the *Department for Computer Science Engineering/ Department for Computer Science and Information Technology/Department of Artificial Intelligence and Data Science* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

Billakanti Siri - Roll No.: 2520030601
Bhavana Kondakrindi - Roll No.:2520030082
Aniketh Pani- Roll No.:2520030182

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	
2.	Introduction	
3.	Literature survey	
4.	Problem Statement and Objectives	
5.	Hardware and Software requirements	
6.	Implementation	
7.	Results & Discussion	
8.	Conclusion & Future Scope	
9.	References	

1.ABSTRACT

DOCS_SIMILARITY is a document similarity analyzer designed to compare multiple documents and identify duplicate or highly similar textual content. The system accepts TXT, PDF, and DOCX files, extracts their text, normalizes it, and represents the content as a sequence of word-level integer tokens. It then constructs a Suffix Array using the prefix-doubling technique and computes the Longest Common Prefix (LCP) Array using the Kasai Algorithm.

The resulting suffix and LCP structures are used to identify common word sequences across different documents. Overlapping matches are merged to reduce repeated counting, after which the system calculates a similarity score for each document pair and ranks the pairs by score. A React-based frontend presents uploaded documents, ranked results, and matching textual segments, while a Java Spring Boot backend performs document extraction and algorithmic processing.

The project demonstrates how core Data Structures and Algorithms concepts can be applied to a practical document-analysis task. It can support preliminary duplicate-document identification and textual-overlap screening. However, it focuses on normalized word-level overlap and does not determine semantic equivalence or conclusively establish plagiarism.

Team Number: 07

Team Members:

1. Billakanti Siri - Roll No.: 2520030601
2. Bhavana Kondakrindi - Roll No.:2520030082
3. Aniketh Pani- Roll No.:2520030182

2. INTRODUCTION

DOCS_SIMILARITY is a document similarity analyzer developed to identify duplicate or highly similar textual content across multiple documents. Comparing large documents manually can be time-consuming, especially when they contain repeated passages. This project addresses the problem by using string-processing algorithms to identify common word sequences and calculate similarity scores.

The system uses **Suffix Arrays** and the **Kasai Algorithm** for constructing the Longest Common Prefix (LCP) Array. The documents are processed at the word level, allowing the system to identify shared textual sequences between different files. Based on the detected overlap, document pairs are assigned similarity scores and ranked from highest to lowest similarity.

The application supports TXT, PDF, and DOCX files. It uses a **Java Spring Boot backend** for document processing and algorithm implementation, along with a **React and Vite frontend** for document uploading and displaying results.

The project demonstrates the practical application of Data Structures and Algorithms concepts to document comparison. It can support duplicate-document identification and preliminary plagiarism screening, although it focuses on textual overlap rather than semantic similarity.

3. LITERATURE SURVEY

Existing research explores Suffix Arrays, LCP structures, document retrieval, and plagiarism detection. These studies provide the theoretical foundation for DOCS_SIMILARITY.

1. Manber and Myers (1993) — Suffix Arrays: A New Method for On-Line String Searches

Introduced Suffix Arrays for efficient string searching. This work provides the foundation for organizing document text and identifying common sequences.

2. Kasai et al. (2001) — Linear-Time Longest-Common-Prefix Computation in Suffix Arrays and Its Applications

Introduced an efficient algorithm for constructing the LCP Array in linear time. DOCS_SIMILARITY uses the Kasai Algorithm to identify common word sequences.

3. Gagie et al. (2017) — Document Retrieval on Repetitive String Collections

Explored suffix-based techniques for retrieving information from document collections containing repeated content. This supports the project's multi-document analysis approach.

4. Louza et al. (2020) — gsufsort: Constructing Suffix Arrays, LCP Arrays and BWTs for String Collections

Studied efficient construction of suffix-based structures for string collections, providing background for processing multiple documents.

5. Baba et al. (2017) — Plagiarism Detection Using Document Similarity Based on Distributed Representation

Explored document-similarity techniques for plagiarism detection, demonstrating the relevance of textual comparison to identifying potentially duplicated content.

6. Hussain and Suryani (2015) — On Retrieving Intelligently Plagiarized Documents Using Semantic Similarity

Investigated semantic similarity for detecting plagiarism when wording changes but meaning remains similar. This highlights a limitation of the project's word-level matching approach.

7. Amirzhanov et al. (2025) — Plagiarism Types and Detection Methods: A Systematic Survey of Algorithms in Text Analysis

Reviewed plagiarism-detection methods and their applications, providing broader context for the project's focus on textual overlap.

Literature Summary

The reviewed studies establish the importance of Suffix Arrays and LCP structures for string processing and demonstrate the relevance of document similarity to duplicate detection. DOCS_SIMILARITY applies these concepts to identify common word sequences, calculate similarity scores, and rank document pairs.

4. Problem Statement and Objectives

4.1 Problem Statement

To design and implement a web-based system that compares multiple documents and identifies highly similar textual content using Suffix Arrays, LCP values computed by the Kasai Algorithm, and a document-pair similarity score.

4.2 Problem Description

Manual comparison of multiple large documents is time-consuming and makes repeated passages difficult to identify. DOCS_SIMILARITY automates text extraction, word-level sequence analysis, scoring, and ranking to help users inspect potential textual overlap.

4.3 Main Objective

The main objective is to develop an efficient suffix-based document similarity analyzer that identifies shared textual sequences between documents and ranks document pairs according to their measured similarity.

4.4 Specific Objectives

- Accept multiple supported documents and extract their text.
- Normalize text and represent words using integer token IDs.
- Construct a Suffix Array using the prefix-doubling technique.
- Compute the LCP Array using the Kasai Algorithm.
- Identify common word sequences across document pairs.
- Merge overlapping matched intervals before scoring.
- Calculate pairwise textual-similarity scores.
- Rank document pairs from highest to lowest similarity.
- Present results through a browser-based interface.
- Discuss the algorithms' complexity, strengths, and limitations.

5. HARDWARE AND SOFTWARE REQUIREMENTS

5.1 Hardware Requirements

- A laptop or desktop computer capable of running the Java backend and Node.js frontend.
- Sufficient memory to process the selected documents.
- Sufficient disk space for the project source code, dependencies, and input documents.
- An internet connection for cloning the repository and installing dependencies.

5.2 Software Requirements

Component	Technology	Purpose
Backend Programming Language	Java	Implements document processing and algorithms
Backend Framework	Spring Boot	Provides backend application functionality
Build Tool	Maven	Builds the backend and resolves dependencies
Frontend Library	React	Implements the user interface
Development/Build Tool	Vite	Runs and builds the frontend

PDF Text Extraction	Apache PDFBox	Extracts text from PDF documents
DOCX Text Extraction	Apache POI	Extracts text from DOCX documents
Runtime	Node.js and npm	Installs dependencies and runs the frontend
Version Control	Git	Tracks source-code changes
Repository Hosting	GitHub	Hosts the project repository
Code Editor	Visual Studio Code or compatible IDE	Supports project development
Web Browser	Modern browser	Runs the application interface

6. IMPLEMENTATION

DOCS_SIMILARITY is implemented using a Java Spring Boot backend and a React-based frontend. The backend processes uploaded documents and performs the similarity analysis, while the frontend displays similarity scores, rankings, and common textual sequences.

6.1 Main Modules

1. Document Extraction: Extracts text from TXT, PDF, and DOCX files using appropriate libraries.
2. Text Preprocessing: Normalizes the extracted text, tokenizes it into words, and assigns integer token IDs.
3. Suffix Array and LCP Construction: Constructs the Suffix Array using prefix doubling and computes the LCP Array using the Kasai Algorithm.
4. Similarity Analysis: Identifies common word sequences, merges overlapping matches, calculates similarity scores, and ranks document pairs.

6.2 Similarity Calculation

The system calculates the similarity score using the following formula:

$$\text{Similarity}(A, B) = \frac{2 \times \text{SharedWords}}{\text{Words}_A + \text{Words}_B} \times 100$$

The score represents the percentage of textual overlap between two documents according to the implemented matching method.

6.3 Execution

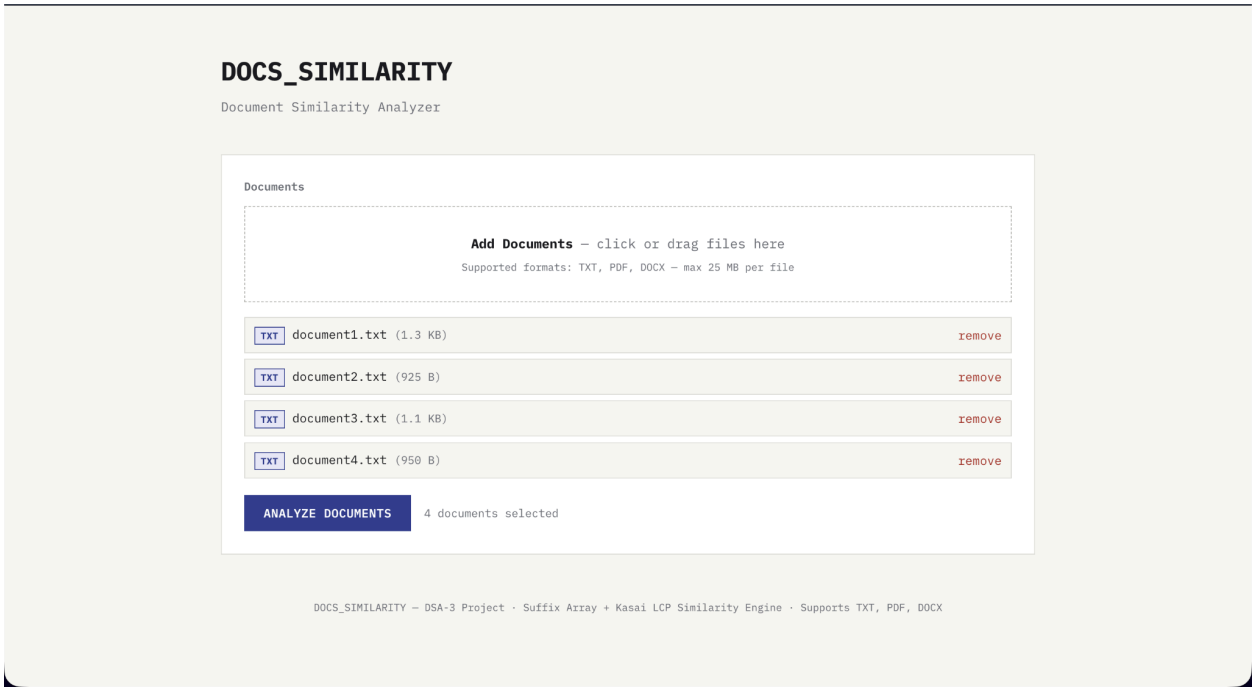
The backend is started using ./mvnw spring-boot:run, and the frontend is started using npm run dev. Users upload documents through the web interface and view the resulting similarity rankings and matching word sequences.

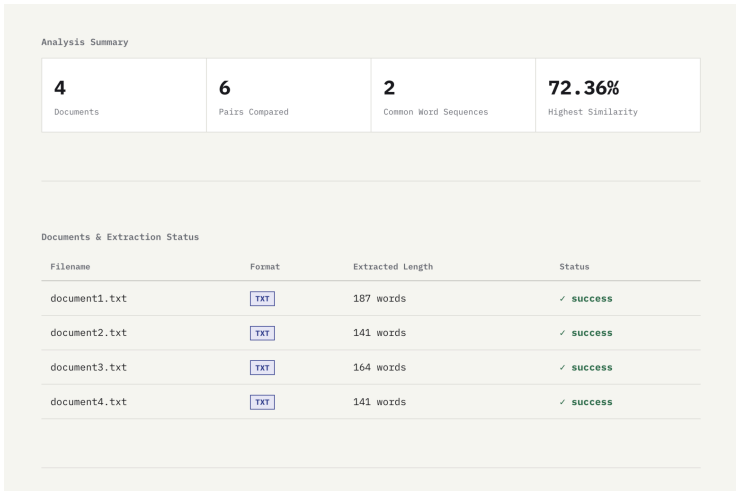
7. RESULTS & DISCUSSION

DOCS_SIMILARITY compares uploaded documents and displays similarity scores, ranked document pairs, and common word sequences. The results help users identify documents containing similar textual content.

Application Screenshots

1. **Upload Interface:**





2. Similarity Results:

Similarity Ranking

Rank	Document A	Document B	Similarity	Common Text
#1	document1.txt	document3.txt	**72.36%**	127 words
#2	document2.txt	document4.txt	**69.50%**	98 words
#3	document1.txt	document2.txt	**0.00%**	0 words
#4	document1.txt	document4.txt	**0.00%**	0 words
#5	document2.txt	document3.txt	**0.00%**	0 words
#6	document3.txt	document4.txt	**0.00%**	0 words

3. Common Sequences:

document1.txt ↔ document3.txt **72.36%**

1 common word sequence found

"renewable energy sources such as solar wind and hydroelectric power are becoming increasingly important as the world seeks to reduce its reliance on fossil fuels solar panels convert sunlight directly into electricity through the photovoltaic effect while wind turbines harness kinetic energy from moving air masses hydroelectric dams generate power by channeling the force of flowing water through turbines connected to generators the transition to renewable energy is driven by both environmental concerns and economic factors as the cost of solar panels and wind turbines continues to fall renewable energy has become competitive with and in many cases cheaper than traditional fossil fuel generation governments around the world have introduced incentives and subsidies to accelerate this transition recognizing the long term benefits of a cleaner energy grid"

Length: 127 words · word offset in document1.txt: 0 · word offset in document3.txt: 11

Algorithm Pipeline

Documents → Tokenization → Token Sequence → Suffix Array → Kasai Algorithm → LCP Array → Common Word Sequences → Similarity Scores → Document Ranking

Algorithm Details

Documents analyzed	4	Combined token count	637 tokens	Suffix Array status	✓ **built**
LCP Array status	✓ **built**	Kasai Algorithm status	✓ **used**	Common word sequences found	2
Document pairs compared	6	Min sequence length	3 words	Suffix Array total entries	637

Suffix Array / LCP — technical view

Showing 300 of 637 entries (limited sample; the full array is used internally for analysis).

SA Index	Suffix Position	Document	Word Offset	LCP	Suffix preview
0	636	(boundary)	-1	0	(END)…
1	494	(boundary)	-1	0	(END)…
2	329	(boundary)	-1	0	(END)…
3	187	(boundary)	-1	0	(END)…
4	0	document1.txt	0	0	renewable energy sources such as solar…
5	341	document3.txt	11	127	renewable energy sources such as solar…
6	65	document1.txt	65	2	renewable energy is driven by both…
7	406	document3.txt	76	62	renewable energy is driven by both…
8	88	document1.txt	88	2	renewable energy has become competitive with

8. CONCLUSION & FUTURE SCOPE

8.1 Conclusion

DOCS_SIMILARITY demonstrates the application of Suffix Arrays and the Kasai Algorithm to document similarity analysis. It identifies common word sequences, calculates similarity scores, and ranks document pairs through a web-based interface.

8.2 Future Scope

The project can be enhanced by adding OCR support for scanned PDFs, semantic similarity detection, improved performance for large documents, and downloadable similarity reports.

9. REFERENCES

1. **Manber, U., & Myers, G. (1993).** ***Suffix Arrays: A New Method for On-Line String Searches.*** **https://epubs.siam.org/doi/10.1137/0222058**
2. **Kasai et al. (2001).** ***Linear-Time Longest-Common-Prefix Computation in Suffix Arrays and Its Applications.*** **https://doi.org/10.1007/3-540-48194-2_17**
3. **Gagie et al. (2017).** ***Document Retrieval on Repetitive String Collections.*** **https://link.springer.com/article/10.1007/s10791-017-9297-7**
4. **Louza et al. (2020).** ***gsufsort: Constructing Suffix Arrays, LCP Arrays and BWTs for String Collections.*** **https://doi.org/10.1186/s13015-020-00177-y**
5. **Baba et al. (2017).** ***Plagiarism Detection Using Document Similarity Based on Distributed Representation.*** **https://doi.org/10.1016/j.procs.2017.06.038**
6. **Hussain & Suryani (2015).** ***On Retrieving Intelligently Plagiarized Documents Using Semantic Similarity.*** **https://doi.org/10.1016/j.engappai.2015.06.007**
7. **Amirzhanov et al. (2025).** ***Plagiarism Types and Detection Methods: A Systematic Survey of Algorithms in Text Analysis.*** **https://doi.org/10.3389/fcomp.2025.1504725**